Gmapping建图

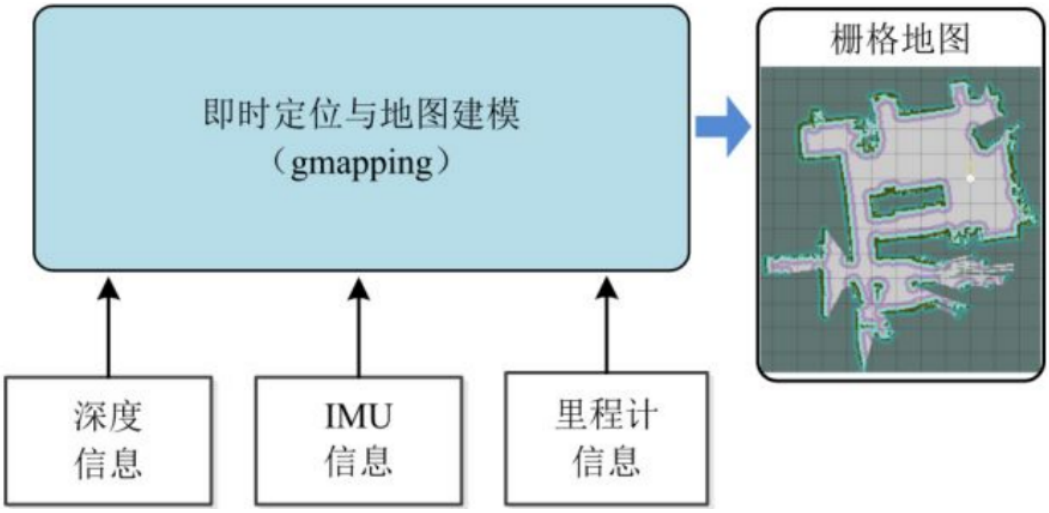
注：树莓派需要与microROS控制板ROS_DOMAIN_ID需要一致，可以查看【MicroROS控制板参数配置】来设置microROS控制板ROS_DOMAIN_ID，查看教程【连接MicroROS代理】判断ID是否一致。

1、Gmapping简介

- gmapping只适用于单帧二维激光点数小于1440的点，如果单帧激光点数大于1440，那么就会出【[mapping-4] process has died】 这样的问题。
- Gmapping是基于滤波SLAM框架的常用开源SLAM算法。
- Gmapping基于RBpf粒子滤波算法，即时定位和建图过程分离，先进行定位再进行建图。
- Gmapping在RBpf算法上做了两个主要的改进：改进提议分布和选择性重采样。

优点：Gmapping可以实时构建室内地图，在构建小场景地图所需的计算量较小且精度较高。

缺点：随着场景增大所需的粒子增加，因为每个粒子都携带一幅地图，因此在构建大地图时所需内存和计算量都会增加。因此不适合构建大场景地图。并且没有回环检测，因此在回环闭合时可能会造成地图错位，虽然增加粒子数目可以使地图闭合但是以增加计算量和内存为代价。



2、程序功能说明

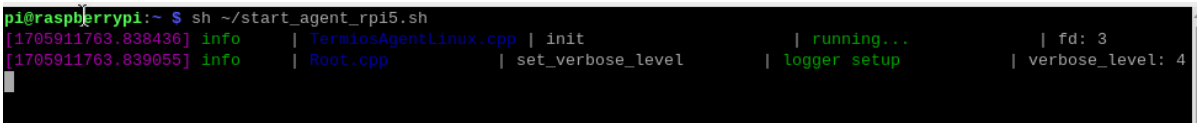
小车连接上代理，运行程序，rviz中会显示建图的界面，用键盘或者手柄去控制小车运动，直到建完图。然后运行保存地图的指令保存地图。

3、查询小车信息

3.1、启动并连接代理

树莓派成功开机后，打开终端输入以下指令打开代理，

```
sh ~/start_agent_rpi5.sh
```



按下microROS控制板的复位按键，等待小车连接上代理，连接成功如下图所示。

```
[1705911851.265754] info     | ProxyClient.cpp    | create_participant       | participant created    | client
_key: 0x6BB64C97, participant_id: 0x000(1)
[1705911851.273538] info     | ProxyClient.cpp    | create_topic             | topic created          | client
_key: 0x6BB64C97, topic_id: 0x000(2), participant_id: 0x000(1)
[1705911851.279639] info     | ProxyClient.cpp    | create_publisher         | publisher created      | client
_key: 0x6BB64C97, publisher_id: 0x000(3), participant_id: 0x000(1)
[1705911851.283998] info     | ProxyClient.cpp    | create_datawriter        | datawriter created     | client
_key: 0x6BB64C97, datawriter_id: 0x000(5), publisher_id: 0x000(3)
[1705911851.289506] info     | ProxyClient.cpp    | create_topic             | topic created          | client
_key: 0x6BB64C97, topic_id: 0x001(2), participant_id: 0x000(1)
[1705911851.294457] info     | ProxyClient.cpp    | create_publisher         | publisher created      | client
_key: 0x6BB64C97, publisher_id: 0x001(3), participant_id: 0x000(1)
[1705911851.299026] info     | ProxyClient.cpp    | create_datawriter        | datawriter created     | client
_key: 0x6BB64C97, datawriter_id: 0x001(5), publisher_id: 0x001(3)
[1705911851.305475] info     | ProxyClient.cpp    | create_topic             | topic created          | client
_key: 0x6BB64C97, topic_id: 0x002(2), participant_id: 0x000(1)
[1705911851.309535] info     | ProxyClient.cpp    | create_publisher         | publisher created      | client
_key: 0x6BB64C97, publisher_id: 0x002(3), participant_id: 0x000(1)
[1705911851.313202] info     | ProxyClient.cpp    | create_datawriter        | datawriter created     | client
_key: 0x6BB64C97, datawriter_id: 0x002(5), publisher_id: 0x002(3)
[1705911851.319437] info     | ProxyClient.cpp    | create_topic             | topic created          | client
_key: 0x6BB64C97, topic_id: 0x003(2), participant_id: 0x000(1)
[1705911851.323740] info     | ProxyClient.cpp    | create_subscriber        | subscriber created     | client
_key: 0x6BB64C97, subscriber_id: 0x000(4), participant_id: 0x000(1)
[1705911851.329366] info     | ProxyClient.cpp    | create_datareader        | datareader created     | client
```

3.2、进入小车docker

打开另一个终端输入以下指令进入docker，

```
sh ros2_humble.sh
```

出现以下界面就是进入docker成功，现在即可通过指令控制小车，

```
pi@raspberrypi:~ $ ./ros2_humble.sh
access control disabled, clients can connect from any host
Successful
MY_DOMAIN_ID: 20
```

4、启动程序

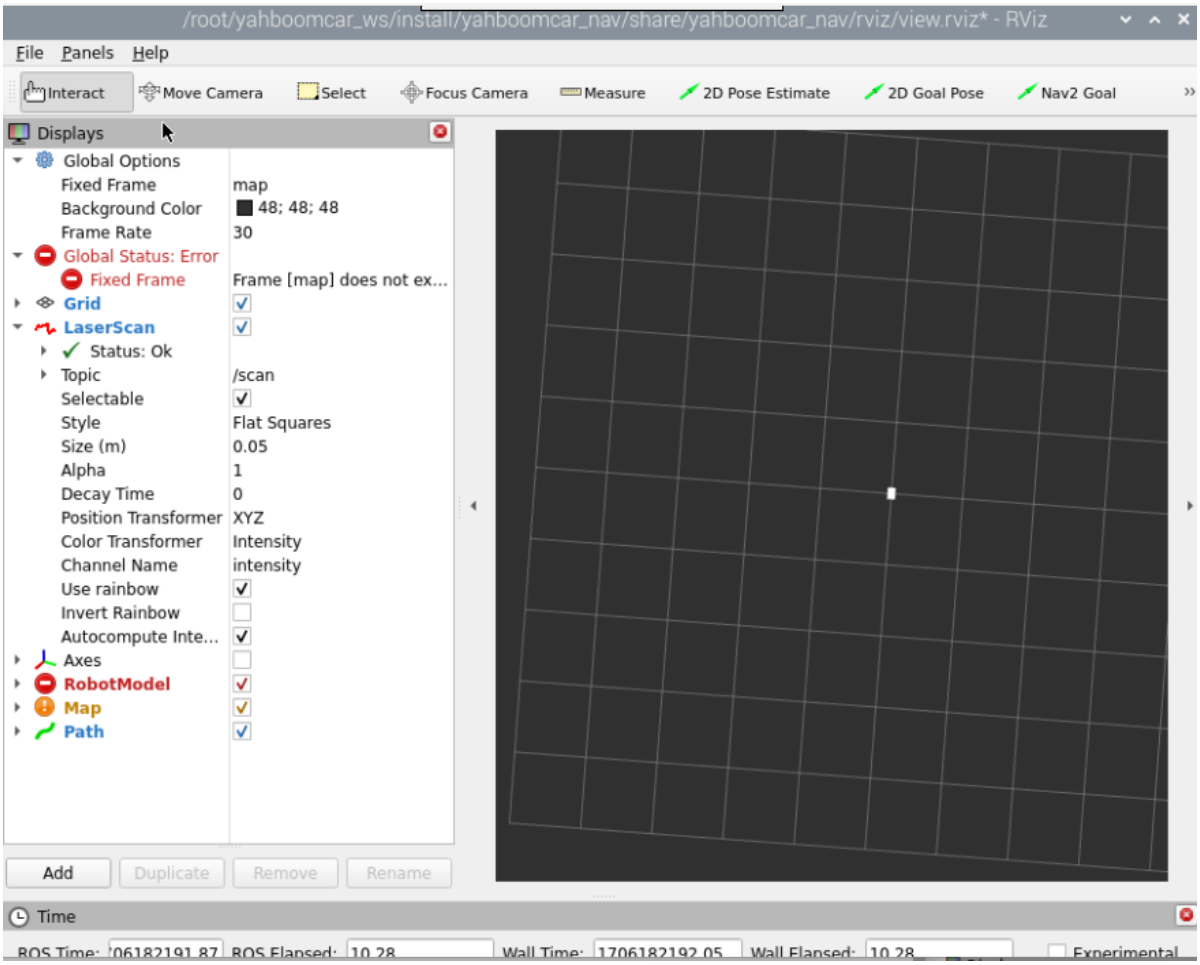
首先启动小车处理底层数据程序，终端输入，

```
ros2 launch yahboomcar_bringup yahboomcar_bringup_launch.py
```

```
[INFO] [imu_filter_madgwick_node-1]: process started with pid [6263]
[INFO] [ekf_node-2]: process started with pid [6265]
[INFO] [static_transform_publisher-3]: process started with pid [6267]
[INFO] [joint_state_publisher-4]: process started with pid [6269]
[INFO] [robot_state_publisher-5]: process started with pid [6271]
[INFO] [static_transform_publisher-6]: process started with pid [6286]
[static_transform_publisher-3] [WARN] [1706181650.342105372] []: Old-style arguments are depreca
ted; see --help for new-style arguments
[static_transform_publisher-3] [INFO] [1706181650.459314055] [base_link_to_base_imu]: Spinning u
ntil stopped - publishing transform
[static_transform_publisher-3] translation: ('-0.002999', '-0.003000', '0.031701')
[static_transform_publisher-3] rotation: ('0.000000', '0.000000', '0.000000', '1.000000')
[static_transform_publisher-3] from 'base_link' to 'imu_frame'
[imu_filter_madgwick_node-1] [INFO] [1706181650.478942143] [imu_filter]: Starting ImuFilter
[imu_filter_madgwick_node-1] [INFO] [1706181650.480114862] [imu_filter]: Using dt computed from
message headers
[imu_filter_madgwick_node-1] [INFO] [1706181650.480197121] [imu_filter]: The gravity vector is k
ept in the IMU message.
[imu_filter_madgwick_node-1] [INFO] [1706181650.480631749] [imu_filter]: Imu filter gain set to
0.100000
[imu_filter_madgwick_node-1] [INFO] [1706181650.480707508] [imu_filter]: Gyro drift bias set to
0.000000
[imu_filter_madgwick_node-1] [INFO] [1706181650.480720249] [imu_filter]: Magnetometer bias value
s: 0.000000 0.000000 0.000000
[static_transform_publisher-6] [WARN] [1706181650.493533858] []: Old-style arguments are depreca
ted; see --help for new-style arguments
[robot_state_publisher-5] [WARN] [1706181650.639387954] [kdl_parser]: The root link base_link ha
s an inertia specified in the URDF, but KDL does not support a root link with an inertia.  As a
workaround, you can add an extra dummy link to your URDF.
[robot_state_publisher-5] [INFO] [1706181650.640061915] [robot_state_publisher]: got segment bas
e_link
[robot_state_publisher-5] [INFO] [1706181650.640174267] [robot_state_publisher]: got segment imu
_Link
[robot_state_publisher-5] [INFO] [1706181650.640191229] [robot_state_publisher]: got segment jq1
_Link
[robot_state_publisher-5] [INFO] [1706181650.640201822] [robot_state_publisher]: got segment jq2
_Link
[robot_state_publisher-5] [INFO] [1706181650.640211952] [robot_state_publisher]: got segment rad
ar_Link
[robot_state_publisher-5] [INFO] [1706181650.640221211] [robot_state_publisher]: got segment yh_
Link
[robot_state_publisher-5] [INFO] [1706181650.640229452] [robot_state_publisher]: got segment yq_
Link
[robot_state_publisher-5] [INFO] [1706181650.640238470] [robot_state_publisher]: got segment zh_
Link
[robot_state_publisher-5] [INFO] [1706181650.640246655] [robot_state_publisher]: got segment zq_
Link
[imu_filter_madgwick_node-1] [INFO] [1706181650.655098332] [imu_filter]: First IMU message recei
ved.
[static_transform_publisher-6] [INFO] [1706181650.681177050] [static_transform_publisher_JarNTEa
1OrW2k0Zb]: Spinning until stopped - publishing transform
[static_transform_publisher-6] translation: ('0.000000', '0.000000', '0.050000')
[static_transform_publisher-6] rotation: ('0.000000', '0.000000', '0.000000', '1.000000')
[static_transform_publisher-6] from 'base_footprint' to 'base_link'
[joint_state_publisher-4] [INFO] [1706181650.989117137] [joint_state_publisher]: Waiting for rob
ot_description to be published on the robot_description topic...
```

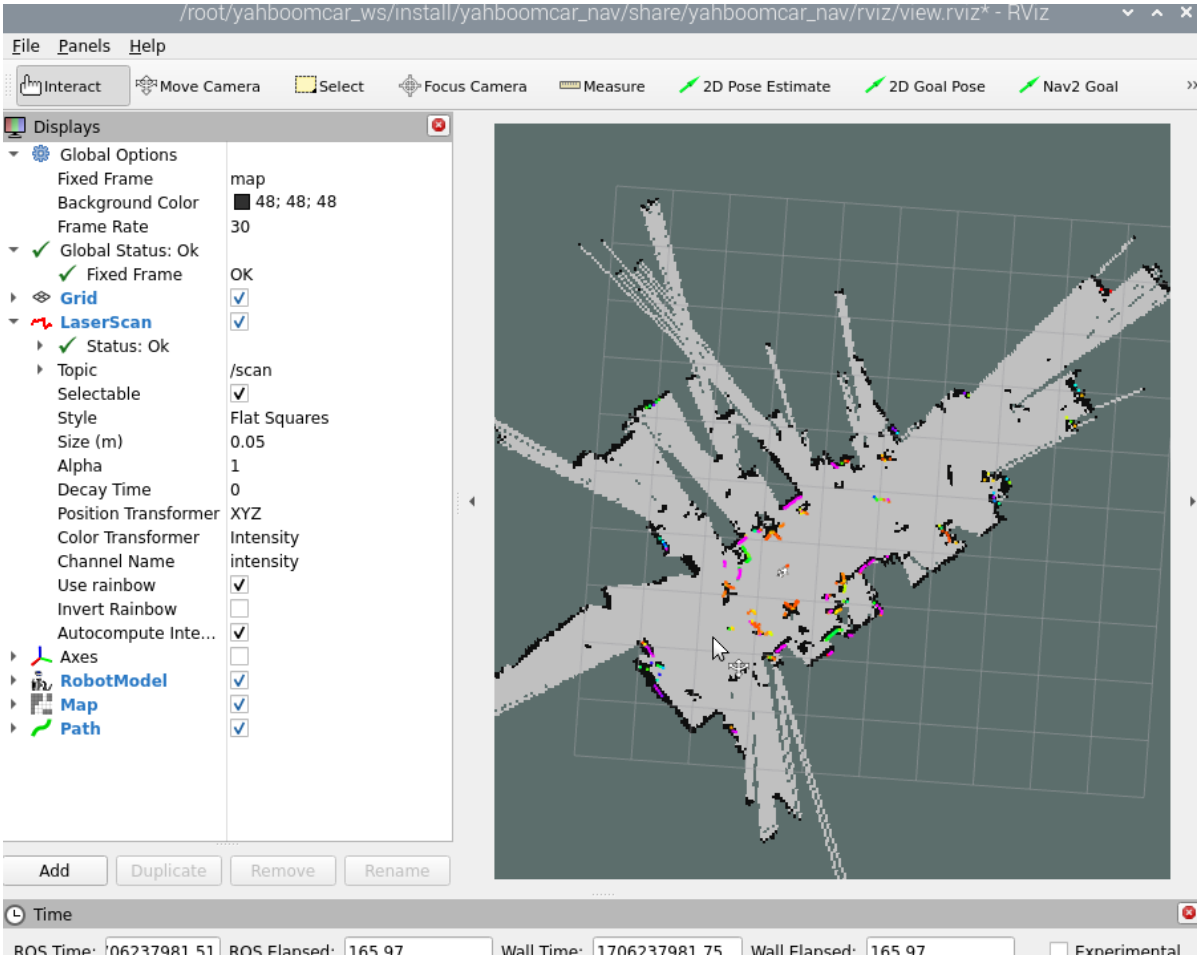
然后，启动rviz，可视化建图，终端输入，

```
ros2 launch yahboomcar_nav display_launch.py
```



此时还没运行建图节点，所以没有数据。接下来运行建图节点，终端输入，

```
ros2 launch yahboomcar_nav map_gmapping_launch.py
```



然后运行手柄控制或者键盘控制，二选一，终端输入，

```
#键盘
ros2 run yahboomcar_ctrl yahboom_keyboard
#手柄
ros2 run yahboomcar_ctrl yahboom_joy
ros2 run joy joy_node
```

然后控制小车，缓慢的走完需要建图的区域，建图完毕后，输入以下指令保存地图，终端输入，

```
root@raspberrypi:~# ros2 launch yahboomcar_nav save_map_launch.py
[INFO] [launch]: All log files can be found below /root/.ros/log/2024-01-26-02-59-04-152767-rasp
berrypi-598
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [map_saver_cli-1]: process started with pid [599]
[map_saver_cli-1] [INFO] [1706237944.412332339] [map_saver]:
[map_saver_cli-1]       map_saver lifecycle node launched.
[map_saver_cli-1]       Waiting on external lifecycle transitions to activate
[map_saver_cli-1]       See https://design.ros2.org/articles/node_lifecycle.html for more inform
ation.
[map_saver_cli-1] [INFO] [1706237944.412495597] [map_saver]: Creating
[map_saver_cli-1] [INFO] [1706237944.412584707] [map_saver]: Configuring
[map_saver_cli-1] [INFO] [1706237944.415206718] [map_saver]: Saving map from 'map' topic to '/ro
ot/yahboomcar_ws/src/yahboomcar_nav/maps/yahboom_map' file
[map_saver_cli-1] [WARN] [1706237944.415269143] [map_saver]: Free threshold unspecified. Setting
 it to default value: 0.250000
[map_saver_cli-1] [WARN] [1706237944.415292550] [map_saver]: Occupied threshold unspecified. Set
ting it to default value: 0.650000
[map_saver_cli-1] [INFO] [1706237946.546319084] [map_saver]: Map saved successfully
[map_saver_cli-1] [INFO] [1706237946.549390831] [map_saver]: Destroying
[INFO] [map_saver_cli-1]: process has finished cleanly [pid 599]
root@raspberrypi:~#
```

```
ros2 launch yahboomcar_nav save_map_launch.py
```

会保存一个命名为yahboom_map的地图，这个地图保存在，

```
/root/yahboomcar_ws/src/yahboomcar_nav/maps
```

会有两个文件生成，一个是yahboom_map.pgm，一个是yahboom_map.yaml，看下yaml的内容，

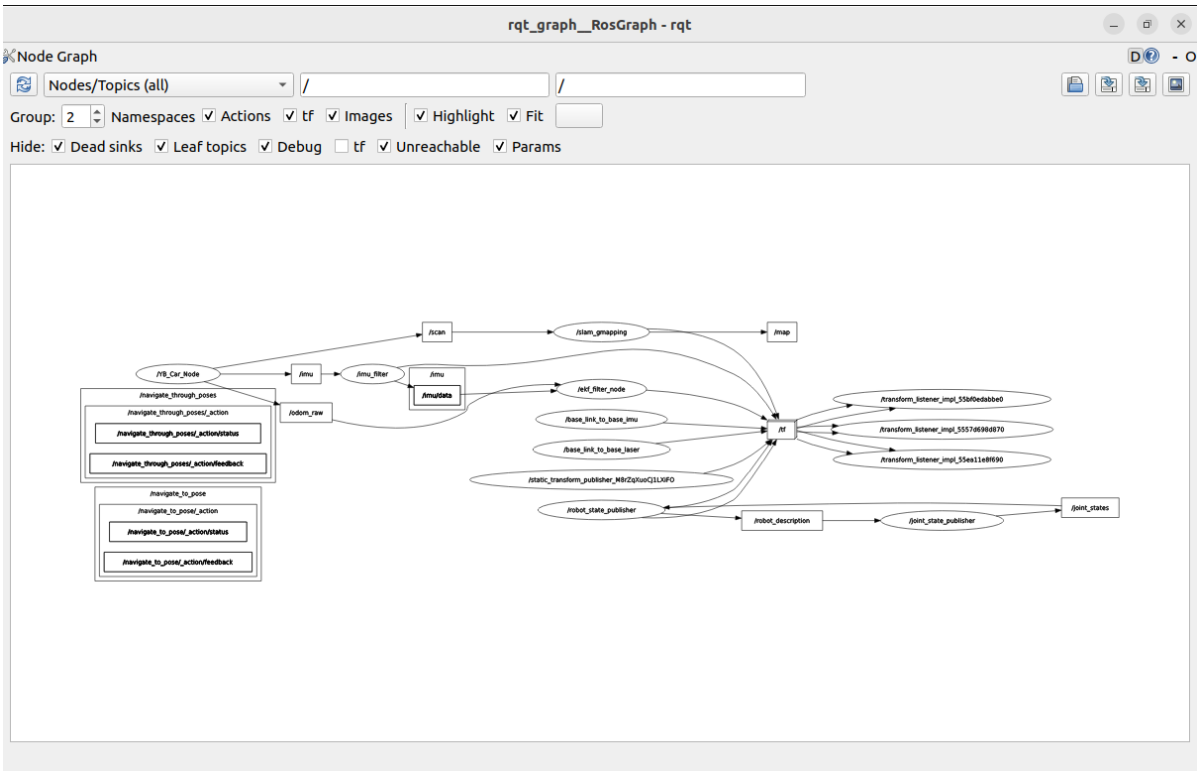
```
image: yahboom_map.pgm
mode: trinary
resolution: 0.05
origin: [-10, -10, 0]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

- image：表示地图的图片，也就是yahboom_map.pgm
- mode：该属性可以是trinary、scale或者raw之一，取决于所选择的mode，trinary模式是默认模式
- resolution：地图的分辨率， 米/像素
- origin：地图左下角的 2D 位姿(x,y,yaw), 这里的yaw是逆时针方向旋转的（yaw=0 表示没有旋转）。目前系统中的很多部分会忽略yaw值。
- negate：是否颠倒 白/黑 、自由/占用 的意义（阈值的解释不受影响）
- occupied_thresh：占用概率大于这个阈值的的像素，会被认为是完全占用。
- free_thresh：占用概率小于这个阈值的的像素，会被认为是完全自由。

5、查看节点通讯图

终端输入，

```
ros2 run rqt_graph rqt_graph
```



如果一开始没有显示，选择【Nodes/Topics(all)】，然后点击左上角的刷新按钮。

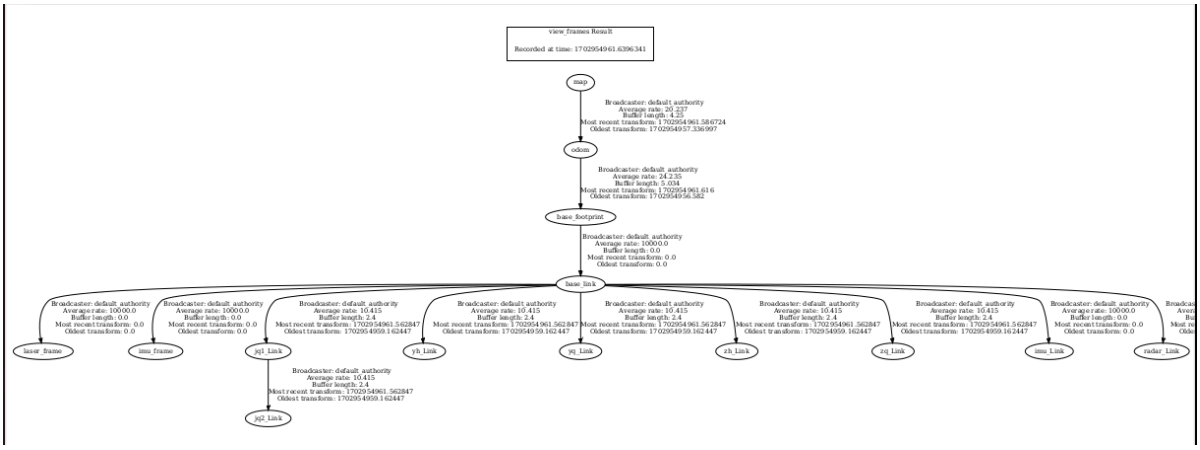
6、查看TF树

终端输入，

```
ros2 run tf2_tools view_frames
```

```
root@raspberrypi:~# ros2 run tf2_tools view_frames
[INFO] [1706240274.802480879] [view_frames]: Listening to tf data for 5.0 seconds...
[INFO] [1706240279.805539455] [view_frames]: Generating graph in frames.pdf file...
[INFO] [1706240279.811022965] [view_frames]: Result:tf2_msgs.srv.FrameGraph_Response(frame_yaml=
"odom: \n  parent: 'map'\n  broadcaster: 'default_authority'\n  rate: 193.930\n  most_recent_tra
nsform: 1706240279.804413\n  oldest_transform: 1706240274.771664\n  buffer_length: 5.033\nimu_fr
ame: \n  parent: 'base_link'\n  broadcaster: 'default_authority'\n  rate: 10000.000\n  most_rece
nt_transform: 0.000000\n  oldest_transform: 0.000000\n  buffer_length: 0.000\nbase_link: \n  par
ent: 'base_footprint'\n  broadcaster: 'default_authority'\n  rate: 10000.000\n  most_recent_tran
sform: 0.000000\n  oldest_transform: 0.000000\n  buffer_length: 0.000\nbase_footprint: \n  paren
t: 'odom'\n  broadcaster: 'default_authority'\n  rate: 19.545\n  most_recent_transform: 17062402
79.773000\n  oldest_transform: 1706240274.810000\n  buffer_length: 4.963\nlaser_frame: \n  paren
t: 'base_link'\n  broadcaster: 'default_authority'\n  rate: 10000.000\n  most_recent_transform:
0.000000\n  oldest_transform: 0.000000\n  buffer_length: 0.000\njq1_Link: \n  parent: 'base_link
'\n  broadcaster: 'default_authority'\n  rate: 10.204\n  most_recent_transform: 1706240279.71725
1\n  oldest_transform: 1706240274.817256\n  buffer_length: 4.900\njq2_Link: \n  parent: 'jq1_Lin
k'\n  broadcaster: 'default_authority'\n  rate: 10.204\n  most_recent_transform: 1706240279.7172
51\n  oldest_transform: 1706240274.817256\n  buffer_length: 4.900\nyh_Link: \n  parent: 'base_li
nk'\n  broadcaster: 'default_authority'\n  rate: 10.204\n  most_recent_transform: 1706240279.717
251\n  oldest_transform: 1706240274.817256\n  buffer_length: 4.900\nyq_Link: \n  parent: 'base_l
ink'\n  broadcaster: 'default_authority'\n  rate: 10.204\n  most_recent_transform: 1706240279.71
7251\n  oldest_transform: 1706240274.817256\n  buffer_length: 4.900\nzh_Link: \n  parent: 'base_
link'\n  broadcaster: 'default_authority'\n  rate: 10.204\n  most_recent_transform: 1706240279.7
17251\n  oldest_transform: 1706240274.817256\n  buffer_length: 4.900\nzq_Link: \n  parent: 'base
_link'\n  broadcaster: 'default_authority'\n  rate: 10.204\n  most_recent_transform: 1706240279.
717251\n  oldest_transform: 1706240274.817256\n  buffer_length: 4.900\nimu_Link: \n  parent: 'ba
se_link'\n  broadcaster: 'default_authority'\n  rate: 10000.000\n  most_recent_transform: 0.0000
00\n  oldest_transform: 0.000000\n  buffer_length: 0.000\nradar_Link: \n  parent: 'base_link'\n
 broadcaster: 'default_authority'\n  rate: 10000.000\n  most_recent_transform: 0.000000\n  oldes
t_transform: 0.000000\n  buffer_length: 0.000\n")
root@raspberrypi:~#
```

运行完毕后，会在终端的目录下生成两个文件分别是.gv和.pdf文件，其中的pdf文件就是TF树。



7、代码解析

这里只说明建图的map_gmapping_launch.py，这个文件路径是，

```
/root/yahboomcar_ws/src/yahboomcar_nav/launch
```

map_gmapping_launch.py

```
from launch import LaunchDescription
from launch_ros.actions import Node
import os
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import PythonLaunchDescriptionSource
from ament_index_python.packages import get_package_share_directory


def generate_launch_description():
    slam_gmapping_launch = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(
        get_package_share_directory('slam_gmapping'), 'launch'),
         '/slam_gmapping.launch.py'])
    )

    base_link_to_laser_tf_node = Node(
     package='tf2_ros',
     executable='static_transform_publisher',
     name='base_link_to_base_laser',
     arguments=['-0.0046412', '0' ,
'0.094079','0','0','0','base_link','laser_frame']
    )

    return LaunchDescription([slam_gmapping_launch,base_link_to_laser_tf_node])
```

这里启动了一个launch文件-slam_gmapping_launch和一个发布静态变换的节点-base_link_to_laser_tf_node。详细看下slam_gmapping_launch，该文件位于，

```
/root/gmapping_ws/src/slam_gmapping/launch
```

slam_gmapping.launch.py，

```
from launch import LaunchDescription
from launch.substitutions import EnvironmentVariable
import launch.actions
import launch_ros.actions
import os
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    return LaunchDescription([
        launch_ros.actions.Node(
            package='slam_gmapping',
            executable='slam_gmapping',
            output='screen',
            parameters=
[os.path.join(get_package_share_directory("slam_gmapping"), "params",
"slam_gmapping.yaml")]),
    ])
```

这里启动了slam_gmapping的节点，加载了slam_gmapping.yaml参数文件，该文件位于（以配套虚拟机为例），

```
/root/gmapping_ws/src/slam_gmapping/params
```

slam_gmapping.yaml

```
/slam_gmapping:
  ros__parameters:
    angularUpdate: 0.5
    astep: 0.05
    base_frame: base_footprint
    map_frame: map
    odom_frame: odom
    delta: 0.05
    iterations: 5
    kernelSize: 1
    lasamplerange: 0.005
    lasamplestep: 0.005
    linearUpdate: 1.0
    llsamplerange: 0.01
    llsamplestep: 0.01
    lsigma: 0.075
    lskip: 0
    lstep: 0.05
    map_update_interval: 5.0
    maxRange: 6.0
    maxUrange: 4.0
    minimum_score: 0.0
    occ_thresh: 0.25
    ogain: 3.0
    particles: 30
    qos_overrides:
      /parameter_events:
```

```yaml
        publisher:
          depth: 1000
          durability: volatile
          history: keep_all
          reliability: reliable
      /tf:
        publisher:
          depth: 1000
          durability: volatile
          history: keep_last
          reliability: reliable
    resampleThreshold: 0.5
    sigma: 0.05
    srr: 0.1
    srt: 0.2
    str: 0.1
    stt: 0.2
    temporalUpdate: 1.0
    transform_publish_period: 0.05
    use_sim_time: false
    xmax: 10.0
    xmin: -10.0
    ymax: 10.0
    ymin: -10.0
```